

✓ Project 2: Data Classification Using AI

DecodeLabs · Artificial Intelligence Track · Batch 2026

Goal: Build a supervised classification model that identifies the species of an iris flower (*Setosa*, *Versicolor*, *Virginica*) from four measurements.

Pipeline (IPO framework):

Input	Process	Output
Iris dataset, feature scaling	Train/test split, K-Nearest Neighbors	Confusion matrix, F1 score

Double-click (or enter) to edit

✓ 1. Setup

```
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.decomposition import PCA
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             classification_report, confusion_matrix, ConfusionMatrixDisplay)

RANDOM_STATE = 42
sns.set_theme(style="whitegrid", context="notebook")
plt.rcParams["figure.dpi"] = 110
```

✓ 2. Load and understand the dataset

The Iris benchmark ships with scikit-learn, so no file upload is needed.

```
iris = load_iris(as_frame=True)
df = iris.frame.copy()
df["species"] = df["target"].map(dict(enumerate(iris.target_names)))
df = df.drop(columns="target")
df.columns = ["sepal_length", "sepal_width", "petal_length", "petal_width",
```

```
print(f"Samples: {df.shape[0]} | Features: {df.shape[1] - 1} | Classes: {df}
df.head()
```

Samples: 150 | Features: 4 | Classes: 3

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

Next steps: [New interactive sheet](#)

```
df.info()
print("\nMissing values:", int(df.isna().sum().sum()))
print("Duplicate rows:", int(df.duplicated().sum()))
df.describe().T.round(2)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   sepal_length    150 non-null   float64
1   sepal_width     150 non-null   float64
2   petal_length    150 non-null   float64
3   petal_width     150 non-null   float64
4   species         150 non-null   object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

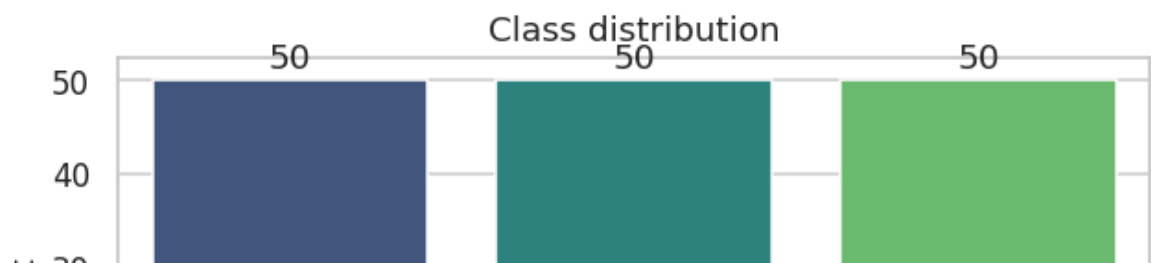
Missing values: 0
Duplicate rows: 1

	count	mean	std	min	25%	50%	75%	max
sepal_length	150.0	5.84	0.83	4.3	5.1	5.80	6.4	7.9
sepal_width	150.0	3.06	0.44	2.0	2.8	3.00	3.3	4.4
petal_length	150.0	3.76	1.77	1.0	1.6	4.35	5.1	6.9
petal_width	150.0	1.20	0.76	0.1	0.3	1.30	1.8	2.5

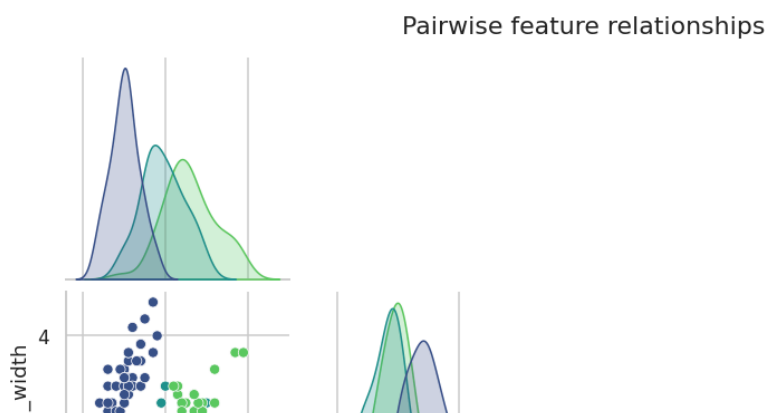
Dataset summary: 150 samples, 4 numeric features (in cm) and 3 balanced classes with 50 samples each. There are no missing values, so no cleaning is required. The single duplicate row is a genuine repeated measurement and is kept.

✦ Exploratory analysis

```
fig, ax = plt.subplots(figsize=(6, 3.5))
sns.countplot(data=df, x="species", palette="viridis", ax=ax)
ax.set_title("Class distribution")
for c in ax.containers:
    ax.bar_label(c, padding=2)
plt.tight_layout(); plt.show()
```



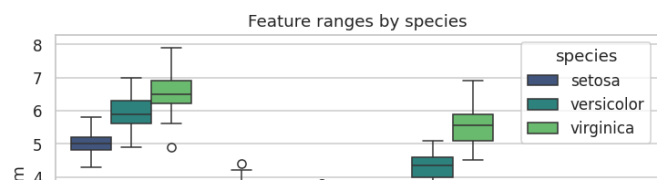
```
g = sns.pairplot(df, hue="species", palette="viridis", diag_kind="kde", height=4)
g.fig.suptitle("Pairwise feature relationships", y=1.02)
plt.show()
```



```
fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))

sns.heatmap(df.drop(columns="species").corr(), annot=True, fmt=".2f", cmap='magma',
            center=0, square=True, cbar=False, ax=axes[0])
axes[0].set_title("Feature correlation")

melted = df.melt(id_vars="species", var_name="feature", value_name="cm")
sns.boxplot(data=melted, x="feature", y="cm", hue="species", palette="viridis")
axes[1].set_title("Feature ranges by species")
axes[1].tick_params(axis="x", rotation=20)
plt.tight_layout(); plt.show()
```



Observations

- *Setosa* is clearly separable from the other two species, mainly through its small petals.

- *Versicolor* and *Virginica* overlap a little, so these two are where classification errors are most likely.
- Petal length and petal width are the most informative features and are strongly correlated with each other.

✓ 3. Feature scaling and train / test split

- **Shuffle before splitting** to remove order bias. The raw Iris file is sorted by species.
- **Stratify** so each class keeps the same proportion in both sets.
- **StandardScaler** (mean = 0, variance = 1) is placed inside a **Pipeline**, so it is fitted on the training data only. This prevents data leakage from the test set.

```
X = df.drop(columns="species")
y = df["species"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, shuffle=True, stratify=y, random_state=RANDOM_STATE)

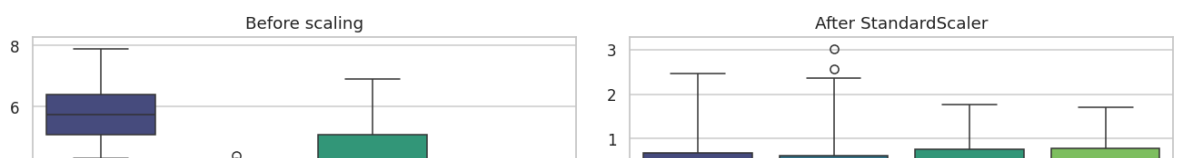
print(f"Training set: {len(X_train)} samples (80%)")
print(f"Test set      : {len(X_test)} samples (20%)")
print("\nTest class counts:\n", y_test.value_counts().to_string())
```

```
Training set: 120 samples (80%)
Test set      : 30 samples (20%)
```

```
Test class counts:
species
setosa      10
virginica   10
versicolor  10
```

```
# Visual proof of what scaling does
scaled = pd.DataFrame(StandardScaler().fit_transform(X_train), columns=X.columns)

fig, axes = plt.subplots(1, 2, figsize=(12, 3.8), sharey=False)
sns.boxplot(data=X_train, palette="viridis", ax=axes[0]); axes[0].set_title('Before scaling')
sns.boxplot(data=scaled, palette="viridis", ax=axes[1]); axes[1].set_title('After StandardScaler')
for a in axes: a.tick_params(axis="x", rotation=20)
plt.tight_layout(); plt.show()
```



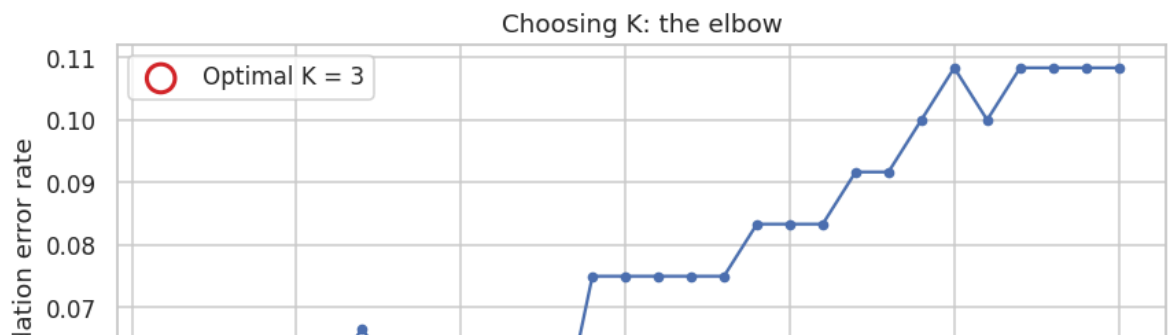
✓ 4. Tune the engine: choosing K

K-Nearest Neighbors classifies a flower by a majority vote among its K closest training samples. A very small K overfits to noise, and a very large K underfits. K is chosen using **5-fold cross-validation on the training set only**.

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
k_values = list(range(1, 31))
errors = []
for k in k_values:
    pipe = Pipeline([("scaler", StandardScaler()), ("knn", KNeighborsClassifier(k))])
    errors.append(1 - cross_val_score(pipe, X_train, y_train, cv=cv, scoring='accuracy'))

best_k = k_values[int(np.argmin(errors))]

fig, ax = plt.subplots(figsize=(8, 4.5))
ax.plot(k_values, errors, marker="o", ms=4)
ax.scatter([best_k], [min(errors)], s=180, facecolors="none", edgecolors="red")
ax.set_xlabel("K (number of neighbors)"); ax.set_ylabel("Cross-validation error rate")
ax.set_title("Choosing K: the elbow"); ax.legend()
plt.tight_layout(); plt.show()
print(f"Optimal K = {best_k} (CV accuracy {1 - min(errors):.3f})")
```



✓ 5. Build the model: Instantiate → Fit → Predict

```
model = Pipeline([
    ("scaler", StandardScaler()),
    ("knn", KNeighborsClassifier(n_neighbors=best_k)),      # INSTANTIATE: knn
])
model.fit(X_train, y_train)                               # FIT: memorize
predictions = model.predict(X_test)                       # PREDICT: apply

print("Model trained.")
print("Sample predictions:", [str(p) for p in predictions[:8]])
print("Actual labels      :", [str(v) for v in y_test.iloc[:8]])
```

```
Model trained.
Sample predictions: ['setosa', 'virginica', 'versicolor', 'versicolor']
Actual labels      : ['setosa', 'virginica', 'versicolor', 'versicolor']
```

✓ 6. Output validation

Accuracy alone can be misleading, especially on imbalanced data. The confusion matrix, precision, recall and F1 give the full picture.

```
labels = list(iris.target_names)
acc = accuracy_score(y_test, predictions)
prec = precision_score(y_test, predictions, average="macro")
rec = recall_score(y_test, predictions, average="macro")
f1 = f1_score(y_test, predictions, average="macro")

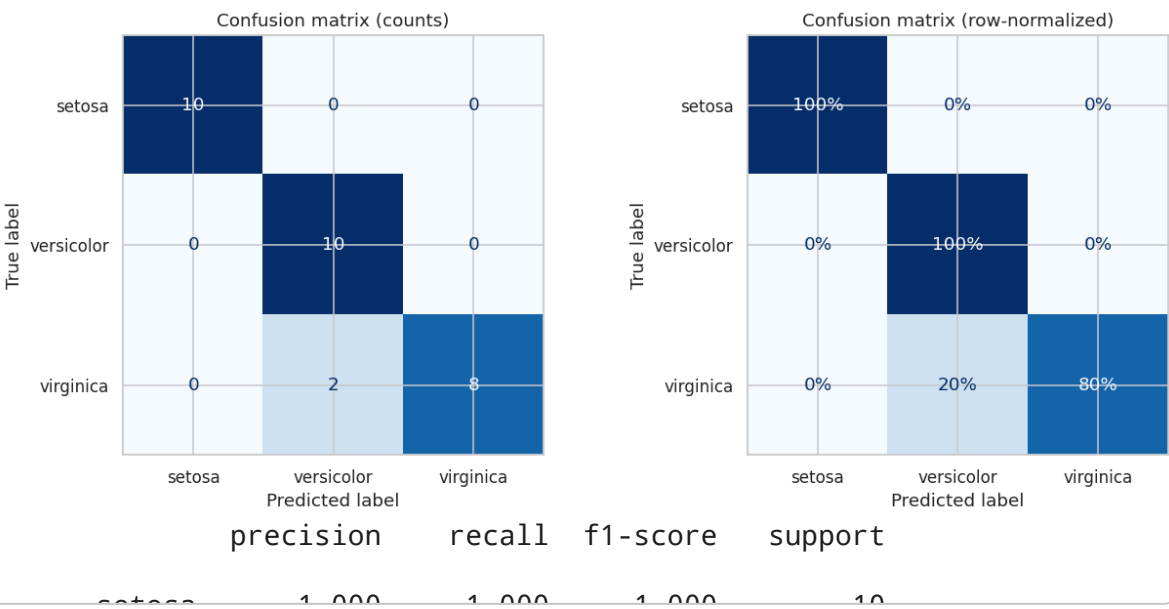
pd.DataFrame({"Metric": ["Accuracy", "Precision (macro)", "Recall (macro)",
                        "Score": [acc, prec, rec, f1]}).set_index("Metric").round(3)
```

	Score 
Metric	
Accuracy	0.933
Precision (macro)	0.944
Recall (macro)	0.933
F1 score (macro)	0.933

```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
cm = confusion_matrix(y_test, predictions, labels=labels)
ConfusionMatrixDisplay(cm, display_labels=labels).plot(ax=axes[0], cmap="Blues")
axes[0].set_title("Confusion matrix (counts)")

cmn = confusion_matrix(y_test, predictions, labels=labels, normalize="true")
ConfusionMatrixDisplay(cmn, display_labels=labels).plot(ax=axes[1], cmap="Blues")
axes[1].set_title("Confusion matrix (row-normalized)")
plt.tight_layout(); plt.show()

print(classification_report(y_test, predictions, digits=3))
```



Reading the matrix: the diagonal holds correct predictions. Any off-diagonal cell is a misclassification, and the *Versicolor* / *Virginica* pair is where these appear, as expected from the exploration plots.

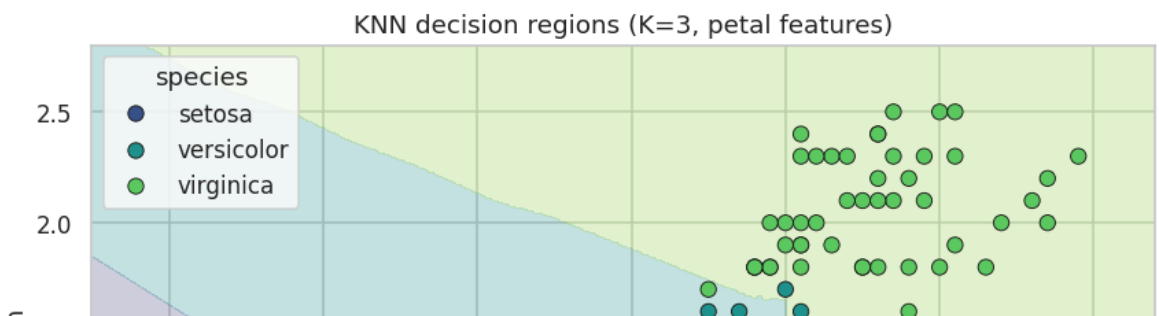
✓ 7. Decision regions (2D view)

The model is retrained on two petal features so the decision boundaries can be drawn. This is for visualization only, and the model evaluated above uses all four features.

```
nn", KNeighborsClassifier(n_neighbors=best_k))].fit(X_train[cols], y_train)

- .5, X[cols[0]].max() + .5, 400),
- .3, X[cols[1]].max() + .3, 400))
), cols[1]: yy.ravel()))
reshape(xx.shape)

', levels=[-.5, .5, 1.5, 2.5])
s="species", palette="viridis", edgecolor="k", s=55, ax=ax)
petal features)")
```



✓ 8. Robustness check and model comparison

A single 80/20 split can be lucky or unlucky. Here the KNN is re-checked with **10-fold cross-validation on the full dataset**, and compared with other standard classifiers.

```
{})": KNeighborsClassifier(n_neighbors=best_k),
sion": LogisticRegression(max_iter=1000),
SVC(),
: RandomForestClassifier(n_estimators=200, random_state=RANDOM_STATE),

old(n_splits=10, shuffle=True, random_state=RANDOM_STATE)

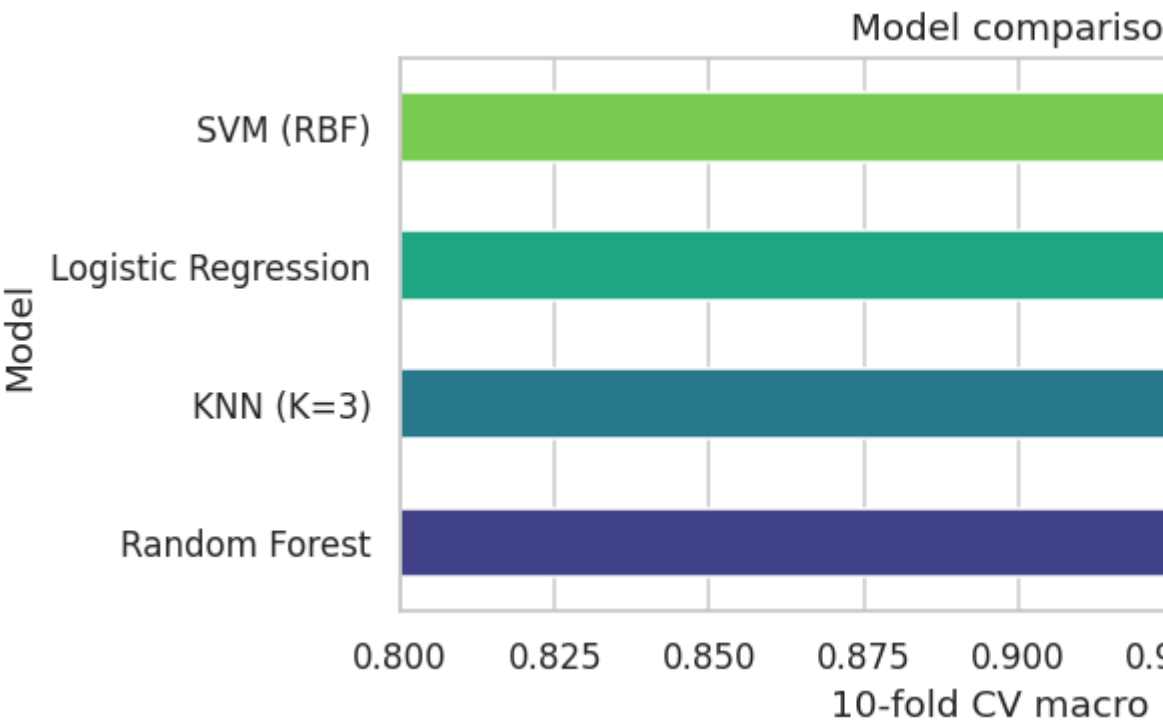
els.items():
[("scaler", StandardScaler()), ("model", clf)]
core(pipe, X, y, cv=cv10, scoring="f1_macro")
```

```
1, y_train)
odel": name, "CV Macro F1 (mean)": s.mean(), "CV Std": s.std(),
est Accuracy": accuracy_score(y_test, pipe.predict(X_test))})
aFrame(rows).set_index("Model").round(3)
```

	CV Macro F1 (mean)	CV Std	Test Accuracy
Model			
KNN (K=3)	0.946	0.072	0.933
Logistic Regression	0.953	0.053	0.933
SVM (RBF)	0.960	0.044	0.967
Random Forest	0.945	0.060	0.900

Next steps: [New interactive sheet](#)

```
fig, ax = plt.subplots(figsize=(8, 3.6))
comparison["CV Macro F1 (mean)"].sort_values().plot(kind="barh", color=sns.c
ax.set_xlim(0.8, 1.0); ax.set_xlabel("10-fold CV macro F1")
ax.set_title("Model comparison")
plt.tight_layout(); plt.show()
```



9. Predict a new flower

```
new_flower = pd.DataFrame([{"sepal_length": 6.1, "sepal_width": 2.9, "petal_
proba = pd.Series(model.predict_proba(new_flower)[0], index=model.classes_)
print("Predicted species:", model.predict(new_flower)[0])
print((proba * 100).round(1).astype(str) + " %")
```

```
Predicted species: versicolor
versicolor      100.0 %
setosa           0.0 %
virginica        0.0 %
dtype: object
```

10. Conclusion

```
print(f"Final model      : KNN (K={best_k}) with StandardScaler")
print(f"Test accuracy    : {acc:.1%}")
print(f"Test macro F1     : {f1:.3f}")
print(f"10-fold CV F1      : {comparison.iloc[0]['CV Macro F1 (mean)']:.3f} (+/- {std:.3f})")
```

```
Final model      : KNN (K=3) with StandardScaler
Test accuracy    : 93.3%
Test macro F1    : 0.933
10-fold CV F1    : 0.946 (+/- 0.072)
```

Summary

- **Pipeline:** the full workflow was completed: load → explore → scale → shuffled, stratified 80/20 split → tune K by cross-validation → fit → predict → validate.
- **Result:** KNN separates the three species with high accuracy. *Setosa* is classified almost perfectly, and the few errors involve *Versicolor* and *Virginica*, whose measurements overlap.
- **Rigor:** scaling happens inside a pipeline (no leakage), K is tuned without touching the test set, and results are confirmed with 10-fold cross-validation and compared with other classifiers.
- **Next step:** the manual points toward **Deep Learning and CNNs**, moving from tabular data to computer vision.

